CREATE VIEW:

```
CREATE VIEW EmployeeDetails AS
SELECT EmployeeID, LastName
FROM Employees
JOIN Departments ON Employees.DepartmentID = Departments.
DepartmentID;
```

CREATE INDEX:

```
CREATE INDEX idx_last_name ON Employees(LastName);
```

ALTER statements

Add primary key:

```
ALTER TABLE Employees
ADD PRIMARY KEY (EmployeeID);
```

Add foreign key:

```
ALTER TABLE Employees
ADD FOREIGN KEY DepartmentID REFERENCES Department(DepartmentID);
```

Add a column:

```
ALTER TABLE Employees
ADD Email VARCHAR(25);
```

Drop a column:

```
ALTER TABLE Employees
DROP COLUMN Email;
```

Add a constraint:

```
ALTER TABLE Employees
ADD CONSTRAINT FK_DepartmentID
FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID);
```

DROP statements

Drop table:

```
DROP TABLE Employees;
```

Drop index:

```
DROP INDEX idx_last_name;
```

Drop view:

```
DROP VIEW EmployeeData;
```

Data manipulation language (DML)

◆ **Data manipulation language:** language that is used to add, modify, delete and retrieve data stored in relational databases.

Data manipulation languages are used to add, modify, delete and retrieve data stored in relational databases.

SQL DML instructions	Explanation
`SELECT`	Retrieves data from one or more tables
`INSERT`	Adds records into a table
`DELETE`	Removes records from a table
`UPDATE`	Modifies existing records in a table

SELECT statements

Retrieve records by displaying specific columns:

```
SELECT field1, field2, field3...
FROM table_name;
```

Retrieve records by displaying attributes that match a given criterion:

```
SELECT field1, field2, ...
FROM table_name
WHERE condition;
```

Retrieve all records:

```
SELECT * FROM table_name;
```

Retrieve records by checking whether specific fields meet specific criteria:

```
SELECT * FROM table_name WHERE condition;
```

A3.3.2 SQL queries between two tables

Including a JOIN in a SELECT statement allows you to aggregate data from multiple tables. For example, considering the employees table and the department table, the script below retrieves the salary expense grouped by department.

JOIN in a SELECT statement

```
SELECT Employees.DepartmentName, SUM(Employees.Salary) AS TotalSalary
FROM Employees
JOIN Department ON Employees.DepartmentID = Department.DepartmentID
GROUP BY Department.DepartmentName;
```

DISTINCT in a SELECT statement

When you want to retrieve unique records and ignore duplicates, you can use the keyword `DISTINCT`.

```
SELECT DISTINCT column1, column2, ...
FROM table_name;
```

HAVING clause vs WHERE clause

The `HAVING` clause is used to filter groups of records created by the `GROUP BY` clause, for example when needing to retrieve the department ID and the average salary per department where the average salary is above 10,000.

The `WHERE` clause is used to filter records *before* grouping, while the `HAVING` clause is used to filter groups *after* aggregation.

```
SELECT DepartmentID, Salary
FROM Employees
GROUP BY DepartmentID
HAVING Salary > 10000;
```

RELATIONAL operators

Relational operators can be used to fetch data that meets specific criteria.

Operator	Example
Equals to	`SELECT * FROM Employees WHERE DepartmentID = 1;`
Not equals to	`SELECT * FROM Employees WHERE DepartmentID <> 1;`
Greater than	`SELECT * FROM Employees WHERE Salary > 10000;`
Smaller than	`SELECT * FROM Employees WHERE Salary < 10000;`
Greater than or equals to	`SELECT * FROM Employees WHERE Salary >= 10000;`
Smaller than or equals to	`SELECT * FROM Employees WHERE Salary <= 10000;`

FILTERING

Operator	Example
BETWEEN filters values between a range	`SELECT * FROM Employees` `WHERE Salary BETWEEN 50000 AND 100000;`
IN filters values that match any value in a given list	`SELECT * FROM Employees` `WHERE DepartmentID IN (1, 2, 3);`
IS NULL filters records with null values	`SELECT * FROM Employees` `WHERE ManagerID IS NULL;`
IS NOT NULL filters records with non-null values	`SELECT * FROM Employees` `WHERE ManagerID IS NOT NULL;`
Combining conditions using logic operators	`SELECT * FROM Employees` `WHERE (DepartmentID = 5 AND Salary > 50000) OR (DepartmentID = 1 AND Salary > 10000);`

Pattern matching

- `LIKE` filters values based on a pattern.
- `%` is used for any sequence of characters (zero or more).
- `_` is used for one single character.

This will retrieve records that start with the letter D:

```
SELECT * FROM Employees
WHERE LastName LIKE 'D%';
```

This will retrieve records that end with the letter d:

```
SELECT * FROM Employees
WHERE LastName LIKE '%d';
```

This will retrieve records that match a specific pattern (start with the letter D, followed by a character, followed by the letter m and followed by zero or more characters):

```
SELECT * FROM Employees
WHERE LastName LIKE 'D_m%';
```

This will retrieve records that are made of three characters:

```
SELECT * FROM Employees
WHERE LastName LIKE '_ _ _';
```

Ordering data

Ordering by a single field:

```
SELECT * FROM Employees
ORDER BY LastName;
```

Ordering by a single field in ascending order:

```
SELECT * FROM Employees
ORDER BY LastName ASC;
```

Ordering by a single field in descending order:

```
SELECT * FROM Employees
ORDER BY LastName DESC;
```

Ordering by multiple fields:

```
SELECT * FROM Employees
ORDER BY DepartmentID, Salary DESC;
```

A3.3.3 SQL update queries

SQL statement	Explanation	Example
`INSERT`	Adds new records	`INSERT INTO table_name (field1, field2, ...)` `VALUES (value1, value2, ...);`
`DELETE`	Deletes records	`DELETE FROM table_name` `WHERE condition;`

`UPDATE`	Modifies records	`UPDATE table_name`
		`SET field1 = value1, field2 = value2, ...`
		`WHERE condition;`

Indexed columns optimize query performance, but updating data in indexed columns may impact performance. When an update is performed for an indexed column, the index needs to be updated as well, which can slow down the operation. Also, index updates can lead to blocking other transactions that need access to a specific record. To overcome those challenges, you can batch the updates to reduce the number of times the index needs to be changed; frequently rebuild or reorganize indexes to reduce fragmentation; or use partial indexes or filtered indexes to limit the scope of the index only to the most relevant records.

A3.3.4 SQL's aggregate functions (HL)

HL ONLY

◆ **Aggregate functions:** functions used to perform calculations on multiple records based on a given field, e.g. AVERAGE, COUNT, MAX, MIN, SUM.

Aggregate functions are used to perform calculations on multiple records based on a given field. Such functions are AVERAGE, COUNT, MAX, MIN, SUM.

Aggregate function	Example
SUM returns the total value of a numerical field	`SELECT SUM(Salary) FROM Employees;`
COUNT returns the number of records that meet the given criteria	`SELECT COUNT(EmployeeID) FROM Employees;`
AVERAGE returns the average value of a numerical field	`SELECT AVG(Salary) FROM Employees;`
MIN returns the smallest value in a field	`SELECT MIN(Salary) FROM Employees;`
MAX returns the largest value in a field	`SELECT MAX(Salary) FROM Employees;`

Aggregate functions on grouped data

Using aggregate functions on grouped data aids reporting and decision-making. An example would be to display the number of employees for each department:

```
SELECT Department.DepartmentName, COUNT(Employees.EmployeeID)
AS ECount
FROM Employees
JOIN Department ON Department.DepartmentID = Employees.DepartmentID
GROUP BY Department.DepartmentName;
```

Another example is when you want to display the average salary for each department:

```
SELECT Department.DepartmentName, AVG(Employees.Salary) AS AvgSalary
FROM Employees
JOIN Department ON Employees.DepartmentID = Department.DepartmentID
GROUP BY Department.DepartmentName;
```

And yet another example would be to display the minimum and maximum salary per department:

```
SELECT DepartmentID, MIN(Salary) AS MinSal, MAX(Salary) AS MaxSal
FROM Employees
GROUP BY DepartmentID;
```

A3.3.5 Database views (HL)

◆ **View:** a virtual table based on the result set of a SELECT query. They do not store data themselves but provide a way to present the data from one or more tables in a customized manner.

A **view** is a virtual table based on the result set of a SELECT query. They do not store data themselves, but provide a way to present the data from one or more tables in a customized manner. Multiple views present different subsets of the data to different users, with the data being presented in different ways according to the user's needs.

There are several advantages of using views:

- **Data complexity hiding:** Views can encapsulate complex queries, simplifying the process for users to query data without needing to understand the underlying SQL.
- **Data consistency:** Views can present data in a consistent manner, even if the underlying tables are modified.
- **Data independence:** The database schema can be changed without affecting the user views.
- **Performance:** Views can increase performance by simplifying complex queries, by abstracting join operations into a single reusable object (the view).
- **Query simplification:** Queries can be simplified by breaking them down into smaller parts, hiding unnecessary details, applying filters and calculations and displaying the results in a view.
- **Read-only or updatable views:** When updating a view, the changes are passed through to the underlying tables from which the view was created, only if certain conditions are met. If those conditions are met, the view is updatable; otherwise, it is read-only. There are three conditions for a view to be updatable: it must be a subset of a single table or another updatable view; all base table fields excluded from the view definition should allow `NULL` values; and the `SELECT` statement of the view should not contain sub-queries (a `DISTINCT` predicate, a `HAVING` clause, aggregate functions, joined tables, user-defined functions or stored procedures).

- **Security:** Views can limit access to specific attributes or records, providing a way to control which data users have access to. There are different types of database views. Some of those types are simple views, complex views or materialized (snapshot) views.

Simple views

Simple views are views based on a single table that do not include complex queries, such as aggregate functions or joins.

An example of a simple view is when displaying some fields from a table. In this case, three fields (`EmployeeID`, `FirstName` and `LastName`) are displayed from the `Employees` table.

```
CREATE VIEW EmpNames AS
SELECT EmployeeID, FirstName, LastName
FROM Employees;
```

Complex views

Complex views are views that include multiple tables and complex queries, such as aggregate functions or joins.

An example of a complex view would be to display data from the `Employees` and `Department` tables.

```
CREATE VIEW EmpDepartment AS
SELECT Employees.EmployeeID, Employees.LastName,
  Department.DepartmentName
FROM Employees
JOIN Department ON Employees.DepartmentID = Department.DepartmentID;
```

Materialized (snapshot) views

Materialized views are views that can be frequently refreshed that store pre-computed data sets derived from a `SELECT` query and stored for later use. As it avoids query re-run (which is used in regular views), it often delivers data faster. The code below is an example of a view that displays the employee ID and their salary for each employee in the `Employees` table.

Creating the snapshot view:

```
CREATE MATERIALIZED VIEW TotalSalaries AS
SELECT EmployeeID, SUM(Salary) AS TotalSal
FROM Employees
GROUP BY EmployeeID;
```

Querying the snapshot view:

```
SELECT * FROM TotalSalaries;
```

A3.3.6 How transactions maintain data integrity in a database (HL)

ACID

Transactions are sequences of SQL operations that are executed as a single unit of work, ensuring data integrity and consistency.

ACID is an acronym that refers to the four properties that define a transaction:

- **Atomicity**

  Transactions are atomic (indivisible and treated as a whole). Either all the actions within a transaction are completed successfully, or none of them is. If any part of the transaction fails, the entire transaction is rolled back, and the database remains unaffected.

- **Consistency**

  Transactions ensure the data follows predefined rules or constraints. The database must be in a valid and expected state after the completion of a transaction.

- **Isolation**

  Transactions are isolated from each other to prevent interference. This ensures that concurrent execution of multiple transactions does not lead to data inconsistencies.

- **Durability**

  Once a transaction is committed and completed successfully, its changes are permanent. Even if the system fails, the changes made by a committed transaction are preserved. Durability is important because transaction data changes must be available even if the database is failing.

Top tip!

When approaching exam questions, ensure you make efficient use of terminology. Often, candidates seem to understand the concepts, but they provide generic responses, with their answers often lacking precision. Terminology must be used precisely when writing responses to gain full marks.

Transaction control language (TCL) commands

TCL commands in SQL are used to manage the transactions. Some of those commands are:

- `BEGIN TRANSACTION`: Used to start a new transaction in SQL.
- `COMMIT`: Used to save all changes made during the current transaction to the database. Once this operation is performed, the changes are permanent and visible to other users.

```
BEGIN TRANSACTION;
UPDATE Employees SET Salary = Salary + 100 WHERE EmployeeID = 1;
UPDATE Employees SET Salary = Salary - 100 WHERE EmployeeID = 5;
COMMIT;
```

In the example above, the `UPDATE` statements for the two records are written and they are both saved once the `COMMIT` statement is reached.

- ROLLBACK: Used to undo all changes made during the current transaction. It reverts the database to the state it was in before the transaction began.

  In the example below, the two UPDATE statements are reversed, and so the table would reach its state prior to the updates being made.

```
BEGIN TRANSACTION;
UPDATE Employees SET Salary = Salary + 100 WHERE EmployeeID = 1;
UPDATE Employees SET Salary = Salary - 100 WHERE EmployeeID = 5;
ROLLBACK;
```

TOK

How has the development of database technology influenced the way we acquire and process knowledge?

The development of database technology has profoundly influenced the ways in which we acquire and process knowledge, touching on key areas such as the nature of knowledge, how it is shared and the ethical considerations involved.

Databases store vast amounts of data, but this raw data only becomes useful when it is processed into information and then interpreted as knowledge. This raises questions about the nature of knowledge itself. How do we distinguish between data, information and knowledge? How does the structure of a database influence what we consider to be true or valuable knowledge?

Databases rely on structured query languages (SQL) and other forms of data communication. The precision and clarity required in database queries contrast with the ambiguity and richness of natural language. This might lead to a more structured, but potentially limited, way of knowing. How does the structured nature of database queries influence our understanding of complex or ambiguous information?

REVIEW QUESTIONS

1. Discuss whether a view is physically stored in a database.
2. Define the term "database transaction".
3. Identify a reason a transaction may need to be rolled back by giving an example.
4. State the effect of rolling back a transaction.
5. Describe the four properties that describe a transaction.